

Odin's Cat: Android-клиент для собственного VPN-контура в эпоху белых списков

Черновик статьи для Хабра

Версия: рабочий драфт. Этот текст написан как стартовая заготовка под большую статью. Его можно сокращать, усиливать примерами, разбивать на блоки и дополнять скриншотами.

Вступление

Обычный VPN всё хуже справляется с реальностью, в которой интернет для пользователя перестаёт быть просто “есть или нет”. Всё чаще возникает другая ситуация: интернет вроде бы работает, мессенджеры открываются, часть сервисов грузится, но доступ к остальному миру либо нестабилен, либо полностью пропадает. На практике это приводит к появлению “белых списков”, когда оператор или сеть пропускает только определённые адреса, платформы и типы трафика.

В такой среде простой сценарий “поднял VPS, поставил VPN и забыл” начинает ломаться. Даже хороший self-hosted сервер может оказаться бесполезным, если до него нельзя нормально достучаться из мобильной сети. Именно из этого практического тупика и выросло приложение **Odin's Cat**: Android-first клиент для собственного VPN-контура, который умеет работать не только с обычным прямым доступом к VPS, но и с промежуточным relay-слоем через Yandex Cloud и HTTP-shaped транспортами.

Идея проекта довольно простая: дать пользователю не публичный “чужой VPN”, а личный, контролируемый контур, который можно развернуть на своём VPS, усилить дополнительной Yandex VM и использовать с Android-клиента через один access-файл. Дальше уже клиент сам помогает поднять нужный путь и не ломать при этом локальные российские сервисы, которые иногда вообще лучше не отправлять через туннель.

В статье я хочу показать не только саму идею, но и практическую реализацию: как устроен клиент, какие есть режимы, зачем нужен Yandex edge, почему используется xhttp, как работает игнорирование локальных сайтов, и как развернуть свой контур на VPS и Yandex VM.

Что такое Odin's Cat

Odin's Cat — это Android-first приложение для подключения к собственному VPN-контру. Оно не пытается быть массовым сервисом “для всех”, а исходит из другой модели:

- у пользователя есть свой VPS;
- при необходимости к нему добавляется отдельная VM в Yandex Cloud;
- доступ в приложение попадает через access/invite JSON;
- клиент на Android поднимает системный VPN и маршрутизирует трафик по выбранной стратегии.

По сути это self-hosted решение для тех, кому нужен не просто VPN, а более гибкий маршрут в условиях нестабильной и ограниченной сети.

Почему обычного VPN уже недостаточно

Проблема не только в блокировке конкретных IP-адресов. На практике ограничения могут работать сразу на нескольких уровнях:

- прямой трафик к VPS просто не проходит;
- TLS-паттерны некоторых клиентов распознаются слишком легко;
- часть платформ из “разрешённой” инфраструктуры остаётся доступной, а остальной интернет нет;
- если отправлять через VPN вообще всё подряд, начинают ломаться локальные сервисы, банки, госресурсы, маркетплейсы и внутрироссийские платформы.

Отсюда и возникает задача не просто “шифровать трафик”, а ****строить маршрут****, который:

1. имеет больше шансов пройти белые списки;
2. не ломает нужные локальные сайты;
3. остаётся под контролем самого пользователя.

Как это работает в общем виде

У Odin's Cat есть несколько уровней маршрутизации.

1. Базовый путь

Самый простой путь — прямое подключение к origin VPS.

Схема:

Android -> VPS -> интернет

Это самый короткий и обычно самый быстрый путь. Но он и самый уязвимый, если сеть плохо пропускает прямые VPN-подключения.

2. Усиленный путь через Yandex edge

Если прямой доступ хуже проходит, добавляется промежуточный edge-слой.

Схема:

Android -> Yandex VM -> VPS -> интернет

Здесь Yandex VM выступает как видимая входная точка. Пользователь не ломает существующий origin-контур, а добавляет поверх него ещё один hop.

3. Whitelist-friendly режим через xhttp

Наиболее интересный режим для текущей архитектуры выглядит так:

Android -> Yandex VM :443 -> xhttp -> origin VPS -> интернет

В этом режиме:

- клиент входит через Yandex Cloud;
- первый hop использует HTTPS-поверхность на `443`;
- дальше трафик до origin идёт через `xhttp`;
- Android-клиент поднимает системный VPN, но часть локальных сайтов можно исключить из туннеля.

Именно этот режим даёт более высокий шанс пройти сеть, где прямой VPN режется жёстче, чем доступ к крупным инфраструктурным платформам.

Почему это может помочь против белых списков

Сразу честно: никакой транспорт не даёт гарантии. Но шансы повышаются за счёт сочетания нескольких факторов.

Видимая входная точка — Yandex Cloud

Для внешней сети первый hop выглядит как обращение к Yandex-инфраструктуре. Это не означает автоматический обход любых ограничений, но даёт более жизнеспособную входную поверхность, чем прямой заход на отдельный VPS.

HTTP-shaped транспорт

xhttp здесь нужен не как “ещё один модный транспорт”, а как способ сделать tunnel-поведение ближе к ожидаемому HTTPS/HTTP-потoku. В условиях DPI и белых списков это может играть заметную роль.

Российские сайты можно не гнать через VPN

Если через туннель отправлять вообще всё, очень быстро начинают страдать сервисы, которые удобнее открывать напрямую. Поэтому в приложении есть отдельный bypass-слой для локальных доменов.

Что умеет клиент

На уровне пользователя важны не только “транспорты”, но и прикладные функции. Сейчас клиент уже умеет:

- импортировать access/invite JSON;
- поднимать системный Android VPN;

- использовать прямой путь и Yandex edge-путь;
- работать с `xhttp`-контуром через `Yandex VM -> VPS`;
- разделять локальные и туннельные маршруты;
- показывать режимы подключения и базовую диагностику;
- использовать bypass для российских сайтов, которым не нужен VPN.

Это позволяет воспринимать приложение не как набор лабораторных скриптов, а как реальный клиент для повседневного использования.

Когда реально включается VPN

Логика клиента довольно практичная:

- внешний и проблемный трафик идёт через VPN;
- часть российских доменов уходит напрямую;
- итоговая цель — не “загнать в туннель всё подряд”, а сохранить рабочий интернет без лишних поломок.

В идеальной картине пользователь подключает профиль один раз, а дальше приложение само держит баланс между проходимостью и совместимостью.

Какие сайты и сервисы мы отправляем в обход VPN

Это один из самых важных блоков, потому что именно он делает повседневное использование более терпимым.

Госуслуги и госсервисы

- `gosuslugi.ru`
- `esia.gosuslugi.ru`
- `lk.gosuslugi.ru`
- `pos.gosuslugi.ru`
- `gov.ru`
- `nalog.ru`
- `fssp.gov.ru`
- `rosreestr.gov.ru`
- `gu-st.ru`

Городские и региональные сервисы

- `mos.ru`
- `pgu.mos.ru`

- `mosreg.ru`
- `gorzdrav.spb.ru`
- `mosenergosbyt.ru`
- `pesc.ru`

Почта, логистика и транспорт

- `pochta.ru`
- `mobileapp.russianpost.ru`
- `rzd.ru`
- `rzd-bonus.ru`
- `avtodor-tr.ru`

Маркетплейсы и ритейл

- `ozon.ru`
- `cdn1.ozonusercontent.com`
- `emex.ru`
- `lemanapro.ru`
- `leroymerlin.ru`

Российские платформы и медиа endpoints

- `yandex.ru`
- `ya.ru`
- `yandex.net`
- `graphql.kinopoisk.ru`
- `fairplay-proxy.ott.yandex.ru`
- `widevine-proxy.ott.yandex.ru`

Прочие домены из практического списка

- `1018213540.rsc.cdn77.org`
- `b2c-ticket-sentry.onelya.ru`
- `bitrix.info`
- `bkvet.ru`
- `cms1.dzvr.ru`
- `counter.yadro.ru`

- `dzvr.ru`
- `reso.ru`
- `showip.net`
- `sys.refocus.ru`
- `vshark.ttk.ru`
- `xn--90aijkdmaud0d.xn--p1ai`

Этот список не является “раз и навсегда идеальным”. Он должен дополняться по практическим кейсам, но уже сейчас показывает ключевую идею: локальные сервисы не обязательно тащить через внешний туннель.

Как устроена инфраструктура

Для минимального рабочего контура используются две точки.

Origin VPS

Это основной сервер, который:

- держит основной сервисный контур;
- остаётся под полным контролем владельца;
- обеспечивает конечный выход в интернет.

Yandex VM

Это дополнительная VM, которая:

- даёт видимый входной edge-слой;
- принимает первый hop от клиента;
- проксирует трафик дальше в origin-контур.

С практической точки зрения важен именно split:

- origin отвечает за стабильность и контроль;
- Yandex edge отвечает за проходимость первого hop.

Почему Android-first

Изначально проект делается с расчётом именно на Android, потому что в мобильных сетях проблема белых списков ощущается особенно болезненно. На десктопе у пользователя часто остаётся больше свободы, а вот на телефоне хотелось добиться сценария:

1. установить APK;
2. импортировать access-файл;

3. нажать Connect;

4. получить рабочий интернет и не убитый при этом локальные сервисы.

Это не отменяет серверной части, но именно Android-клиент делает архитектуру прикладной.

Как развернуть свой контур

Шаг 1. Подготовить VPS

Нужен обычный VPS с публичным адресом и SSH-доступом. Это origin-узел, который будет держать основной серверный контур.

Шаг 2. Подготовить Yandex VM

Для Yandex VM желательно использовать обычный SSH-ключ и локального пользователя, а не усложнённый IAM/OS Login сценарий.

На практике это означает:

- создать SSH-ключ локально;
- при создании VM передать публичный ключ;
- через `cloud-init` создать пользователя с `sudo NOPASSWD`.

Шаг 3. Проверить SSH

До любого деплоя полезно убедиться, что:

- вход на VPS работает;
- вход на Yandex VM работает;
- пользователь имеет нужные права.

Шаг 4. Поднять origin

После этого разворачивается origin-контур на VPS.

Шаг 5. Прикрепить edge

Дальше к origin-контру добавляется Yandex edge.

Шаг 6. Сгенерировать access-файл

Когда серверная часть готова, создаётся access/invite JSON для клиента.

Шаг 7. Импортировать профиль в Android

Пользователь импортирует `.odinone-access.json` в приложение и запускает VPN.

Как пользователь подключается в приложении

С точки зрения конечного пользователя flow должен быть максимально простым:

1. установить APK;
2. импортировать access-файл;
3. выбрать профиль;
4. включить VPN;
5. использовать интернет как обычно.

В идеале читатель статьи должен увидеть, что за всей сложной архитектурой всё ещё стоит очень простой пользовательский сценарий.

Диагностика и типовые проблемы

Этот раздел точно стоит оставить в статье, потому что он очень полезен в практической жизни.

VPN включился, а интернета нет

Проверять:

- поднялся ли системный Android VPN;
- активировался ли нужный runtime, а не старый fallback;
- корректно ли работает DNS внутри туннеля;
- действительно ли edge доходит до origin.

Локальные российские сайты перестали открываться

Обычно это означает, что bypass-правила либо не применились, либо домен ушёл в неправильный resolver path.

Скорость ниже, чем в direct path

Это ожидаемо. Режим через Yandex edge и xhttp обычно покупает проходимость ценой скорости. Но его можно постепенно оптимизировать.

Скорость и компромиссы

Любая whitelist-friendly архитектура платит за свою устойчивость сложностью.

В прямом режиме путь короче:

Android -> VPS

В усиленном режиме путь длиннее:

Android -> Yandex VM -> VPS

А если между hop'ами добавляются ещё и HTTP-shaped слои, неизбежно появляется дополнительный overhead. Поэтому важно честно проговорить:

- да, такой режим может быть медленнее прямого;
- да, он даёт более высокий шанс пройти плохую сеть;
- да, его можно оптимизировать, но обычно не до скорости “голого” кратчайшего path.

Безопасность и модель доверия

Важное достоинство self-hosted подхода в том, что пользователь сам контролирует инфраструктуру. Но из этого вытекает и ответственность:

- нельзя публиковать приватные ключи;
- нельзя светить реальные серверные адреса в UI и скриншотах без необходимости;
- release APK должен быть нормально подписан;
- серверный и публичный release-репозитории стоит разделять.

Что уже реализовано

На текущем этапе проект уже даёт не только идею, но и рабочую реализацию:

- Android-клиент;
- импорт access-файлов;
- системный VPN;
- Yandex edge fast path;
- Yandex camo path;
- `xhttp`-контур через Yandex VM;
- bypass для российских сайтов;
- owner deploy и диагностика.

То есть это уже не просто исследование, а прикладной инструмент, который можно пробовать в реальной среде.

Что ещё хочется улучшить

У проекта остаётся большой запас для развития:

- более умный fallback между direct, fast и camo;
- улучшение transport-tuning;

- удобный SSH/bootstrap onboarding;
- отдельный Windows helper;
- ещё более понятная диагностика для конечного пользователя;
- полевые прогоны в сетях с реальными белыми списками.

Почему это вообще интересно читателю Хабра

Потому что речь не о “ещё одном VPN-клиенте”, а о попытке сделать прикладной, самоуправляемый и повторяемый доступ к сети в среде, где обычные подходы уже не работают так, как раньше.

Тут есть сразу несколько интересных пластов:

- Android-клиент и UX поверх сложной сетевой логики;
- transport layer и маршрутизация;
- self-hosted инфраструктура;
- edge/origin split;
- практический bypass локальных сервисов;
- компромисс между скоростью и проходимостью.

Заключение

Если интернет постепенно превращается в среду с фрагментированным доступом, одних только “традиционных VPN-настроек” становится мало. Нужна более сложная, но при этом прикладная архитектура: со своим origin, с промежуточным edge-слоем, с правильным клиентом и с маршрутизацией, которая умеет не ломать локальную повседневность.

Именно такую задачу и пытается решить ****Odin's Cat****. Это не серебряная пуля и не гарантия прохождения любой сети, но это уже рабочий набор инструментов, который даёт пользователю больше контроля и больше шансов сохранить доступ к нормальному интернету.

Что ещё можно добавить в финальную версию статьи

- реальные скриншоты приложения;
- схему архитектуры;
- мини-диаграмму `Android -> Yandex -> VPS`;
- пример access-файла без секретов;
- пример cloud-init для Yandex VM;
- короткий troubleshooting checklist;

- блок “какие кейсы реально прошли в полях”.